Team Members: Nate Heath, Elliot Warner, William Thomas

Project Title: Data-Day

Goal Statement: To build and open source a 64 x 32 pixel desktop display and the developer platform for creating applications for the device.

Components: This document will detail the Dial Button, Orange Pi, and Display components of the D0 high level system. The Device Application and Upload Server components were completed in the D1 detailed design.

Component 1 – Dial Button

Our product will have two dial buttons. One dial button will be used to control overall application switching: turning the dial button will rotate the display between the different uploaded user applications. The second dial button input will be available to the user programs. Code can be written using our exposed API endpoints to define how to handle input from the second dial button.

Data Model: No persistent data

Behavior:

ALGORITHM: ApplicationSelectionController

INPUTS:

- List of user-submitted applications
- Physical dial rotation events (clockwise or counterclockwise)
- Application submission and removal events
- Application execution status

OUTPUTS:

- Updated application selection
- LED display updates
- Application launch or switch requests
- Error messages and logs

BEGIN

1. Initialize the application manager, dial input handler, and LED display driver.
2. Retrieve the list of installed and valid user-submitted applications.
3. Set the selected application index to 0.

4. IF no applications are available:

    - Display a "No Applications Available" message.
    - Wait for an application to be submitted.

ELSE:

    - Display the first available application.

5. WHILE the device is running:

a. Check for application submission or removal events.

b. IF the application list has changed:

    - Refresh the list of available applications.
    - Ensure the selected application index is valid.
    - IF no applications remain:
        - Display a "No Applications Available" message.
    - ELSE:
        - Update the displayed application if necessary.

c. Check for physical dial rotation events.

d. IF the dial rotates clockwise:

    - Move the selected application index to the next application.
    - IF the end of the application list is reached:
        - Wrap around to the first application.
    - Update the LED display to show the selected application.

e. ELSE IF the dial rotates counterclockwise:

    - Move the selected application index to the previous application.
    - IF the beginning of the application list is reached:
        - Wrap around to the last application.
    - Update the LED display to show the selected application.

f. IF an application is selected for execution:

- Request the application manager to activate the selected application.
- Stop the previously active application if necessary.
- Launch the selected application.
- Record and handle any execution errors.

g. Continue monitoring dial input and application status.

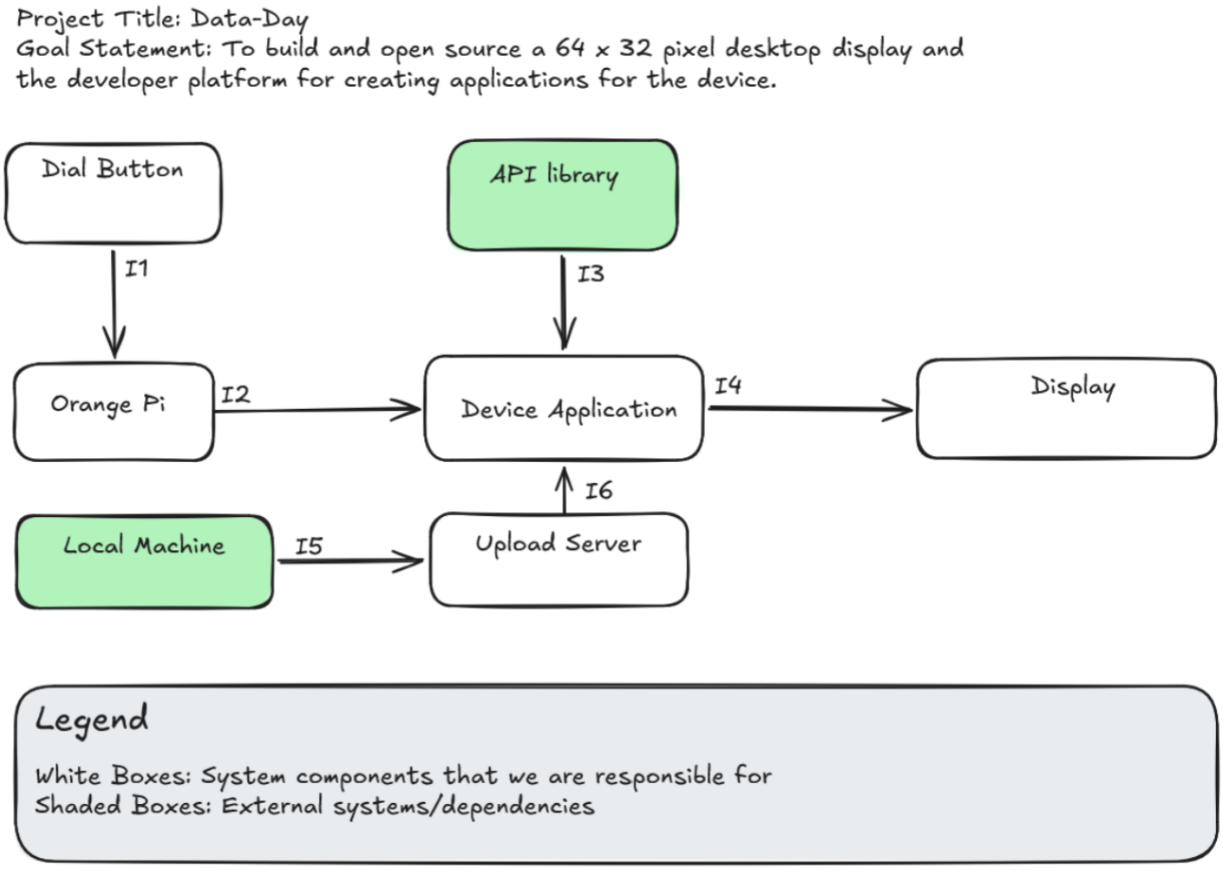
6. Stop the active application when shutdown is requested.
7. Clear the LED display or set it to a safe state.
8. Release resources used by the application manager, dial input handler, and display driver.

END

Component 2 – Orange Pi

The Orange Pi Zero 3 is the “brain” of our product. It hosts the server where users will upload their own program files. It also handles the switching of user programs. Lastly, it runs the user programs and passes the program output to the LED display board.

Data Model:



Behavior:

ALGORITHM: OrangePiMainController

INPUTS:

- Application upload requests
- Web management requests
- Physical dial/button events
- Application output and status
- System shutdown signal

OUTPUTS:

- LED display updates
- Application status responses
- Installation and execution results
- Error messages and logs

BEGIN

1. Boot the operating system.
2. Initialize the display driver, framebuffer, application manager, and network service.
3. If initialization fails:
   - Log the error.
   - Display an error indication if possible.
   - Retry recoverable initialization failures.
   - Otherwise enter a safe error state.
4. Discover installed applications. Validate their manifests and entry points.
5. Select the default application or show an application-selection screen.
6. Start the web management service.
7. WHILE shutdown has not been requested:
   a. Check for a web request.
   b. IF an application upload is received:
      - Validate the package.
      - Reject invalid or unsafe package structures.
      - Install the application if valid.
      - Return success or an error response.

c. Check for application-management requests. Launch, stop, delete, or list applications as requested.

d. Check for physical dial/button input. Update the selected application. If selection is confirmed, switch applications.

e. Monitor the running application.

IF it crashes or exceeds configured limits:

- Stop it if possible.
- Record the failure.
- Return to the selection screen or launch a fallback application.

f. Render the active application's output.

g. Continue refreshing the LED matrix.

8. Stop the running application gracefully.
9. Clear or set the display to a safe state.
10. Stop services and release resources.

END

Component 3 – Display

The 64 x 32 pixel LED display is the user-facing side of our product. It is responsible for clearly communicating to the user whatever it has been programmed to, from the current weather outside to the sports scores for today.

Data Model: No persistent data

Behavior:

ALGORITHM: LEDDisplayController

INPUTS:

- Display initialization request
- Application drawing commands
- Framebuffer updates
- Brightness adjustment requests
- Display shutdown signal

OUTPUTS:

- LED matrix pixel updates
- Display initialization status
- Display operation status
- Display error messages and logs

BEGIN

1. Initialize the LED display driver.

2. Configure the display resolution to 64 × 32 pixels. Configure the required GPIO pins and display settings.

3. Allocate and initialize the framebuffer.

    - Set all pixels to the default background color.

4. IF display initialization fails:

    - Log the error.
    - Report the display as unavailable.
    - Return an initialization failure status.

5. WHILE shutdown has not been requested:

    a. Receive drawing commands from the application or rendering engine.

    b. IF a clear command is received:

    - Set all framebuffer pixels to the specified background color.

    c. IF a pixel, text, shape, or image command is received:

        - Validate the command parameters.
        - Update the corresponding framebuffer pixels.

    d. IF a framebuffer update is received:

        - Validate the framebuffer dimensions and pixel data.
        - Prepare the new frame for display.

    e. IF a brightness adjustment is requested:

        - Validate the requested brightness level.
        - Apply the brightness setting.

    f. IF a valid frame is ready:

- Send the framebuffer to the display driver.
- Update the LED matrix output.

g. Continue refreshing the LED matrix at the rate required by the hardware driver.

h. IF a display operation fails:

- Log the error.
- Report the failure to the caller.
- Attempt recovery if supported.

6. Clear the display or set it to a safe output state.
7. Stop the display refresh process.
8. Release framebuffer memory and display resources.
9. Shut down the display driver.
10. Report successful shutdown.

END

Edge Cases

ID	Component	Category	Edge Case	Design Decision
EC01	Dial Button	Boundary values	User rotates dial beyond available application options.	Dial wraps around to opposite end of the application list.
EC02	Dial Button	Timing/concurrency	User rotates dial rapidly.	Implement input debouncing so rapid rotation does not cause missed/duplicated selections.
EC03	Dial Button	Bad/missing input	Dial produces invalid signal.	Validate input events and ignore invalid signals.
EC04	Dial Button	Ext. dependencies	Dial button loses connection or Orange Pi cannot communicate with the dial button.	Detect communication failures and log errors.
EC05	Orange Pi	Boundary values	User uploads an application that	Define maximum application sizes and check available

			exceeds allowed file size.	resources before installation. Reject uploads that exceed limits and return an error message.
EC06	Orange Pi	Timing/concurrency	One application is running while another is uploaded or user switches to another application.	Call saving logic defined in user application before moving to other application. Allow uploads to occur in the background.
EC07	Orange Pi	Bad/missing input	An uploaded application has an invalid entry point or malformed data.	Validate application before running.
EC08	Orange Pi	Ext. dependencies	Wi-Fi disconnects, external API becomes unavailable.	Continue running locally installed applications when network access is lost.
EC09	Display	Boundary values	Application attempts to draw to pixels outside 64 x 32 display.	Define valid pixel coordinates in SDK. Reject out-of-bounds draw attempts.
EC10	Display	Timing/concurrency	Applications updates framebuffer while Orange Pi is sending current frame to LED panel.	Keep hardware refresh timing separate from application drawing operations.
EC11	Display	Bad/missing input	Renderer sends a framebuffer with invalid dimensions or pixel values	Validate dimensions and pixel values before passing to display. Save last valid frame and reject invalid frames.
EC12	Display	Ext. dependencies	LED panel loses power or connection fails.	Detect failures when possible and report status to Orange Pi. Develop safe recovery procedure.

Requirements Traceability

ID	Description	Priority	Component	Explanation
REQ1	Allow User to Download Software onto Device	HIGH	Orange Pi / React Frontend	The user can download and flash the device software from the frontend
REQ2	Allow User to Download Application for Device	HIGH	Orange Pi / React Frontend / Api	The user can connect to the Pi via the internet and Download Apps to the Pi by calling the API that hosts our apps
REQ3	Display software on Screen	HIGH	Orange Pi / Hub75 Board	Using the Orange Pi we connect to the Hub75 board and provide input for the Hub75 Board
REQ3	Hosting Application and Software Server	HIGH	Vercel or other Provider	We'll use Vercel to host our frontend to hold all downloadable Packages and Render to make them downloadable
REQ4	Orange Pi to Hub75 library	HIGH	Orange Pi / Hub75	We need to create a library to standardize inputs to the hub 75 board since there are no current libraries for connecting them
REQ5	Handle Initialization Errors	HIGH	Orange Pi / Logging and status script	Write a script that will handle passing the system and logging / displaying in the UI if the booting errors
REQ6	Implement Dial navigation	MID	Dial / Orange Pi /	We're going to write into the SDK the code to handle the

			Developer SDK	**dial addition and that script will live on the PI**
REQ7	**Handle Application Errors**	**MID**	**Orange Pi / Error code**	**We must write in extensive error handling into our sdk to inform the dev what's wrong**
REQ8	**FastAPI Backend for API**	**HIGH**	**Render or other provider**	**We need a backend to host our API to connect to our stored files so that users can download and upload apps**
REQ9	**App management / download**	**MID**	**Github / Orange Pi / Frontend**	**We need to set up a way to link to apps to our website so users can download to the Pi from the github using API calls**
REQ10	**App verification and rating**	**LOW**	**Frontend / Github**	**Create a system for verifying if apps are safe and how they rank amongst the community**

Design Rationale

1. **We chose the Orange Pi Zero 3 over the Raspberry Pi Zero 2W because** of the **lack of availability for the Raspberry P**i. There are currently board shortages, so the prices of these devices have gone a lot. We accept that they might not be as high of quality for the lower cost.
2. **We chose the 64x32 HUB75 RGB LED Matrix over an E-Ink display because** the device is cheaper. We accept that it will be harder to display more detailed data for the increase in creativity that comes with the limited display.
3. **We chose React for the local web upload interface over plain HTML/JS because** it's a well-documented and easy to build in frontend framework. We accept that it might not scale very well because we don't need a large platform for the scope of this project.

4. **We chose a physical dial with a button over standard buttons because** of the feedback and increased options. We accept that there are less buttons overall, but we like the creativity that comes with using a wheel for input.
5. **We chose a 64x32 LED display over other sizes** because the device will have a better form factor for desks and similar applications. We accept the more limited display size because the saved placement area allows for more flexible placement.
6. **We chose Python over C++ for the user-submitted app language because** Python is a more common and easy to understand language. We accept that the software will run slower because the increase in development time will encourage users to design more apps.

Peer Review Notes

ID	Rubric Criterion	Issue & Reviewing Team	Action	Owner	Priority	Due Date
PR-01	Risk	The security algorithm removes suspicious lines from uploaded Python programs. Could this break otherwise valid programs? — Team: Nate Poppe and Braden Hayes	Consider rejecting flagged uploads with an explanation rather than modifying source code.	William Thomas	Worth Considering	Week 9
PR-02	Completeness	When users switch applications using the dial, does the previous application continue running or terminate? How is	Define application lifecycle behavior, including execution, state	Elliot Warner	Must-fix	Week 9

		its state handled? — Team: Nate Poppe and Braden Hayes	**preservation, and hardware resource management.**			
PR-03	**Completeness**	**Component Orange Pi 3** **Since you guys are choosing to make this open source, is there any plan on creating some type of library for application to interaction with the IO's and Display** **Team: Cameron Pocisk, Jake Skaff**	**Define what capabilities can come out of the box with the device and basic implementation with clear documentation**	**William Thomas**	**Important**	**Week 10**
PR-04	**Completeness /Risk**	**API: /api/v1/program/upload HTTP/1.1** **Is there a protocol needed for large python files?** **Team: Cameron Pocisk, Jake Skaff**	**Clarify how large files will be handled, and managed with AST**	**Nate Heath**	**Must-fix**	**Week 10**

Completeness drew the most feedback across the reviews, appearing in 3 out of the 4 items, mainly focusing on missing documentation/system specifications.

Top two issues raised:

1) **ColorStylus:** There was an issue with their storage silently being overwritten once they reached a maximum capacity, we suggested for them to add an indicator to their light display to notify the user that they are almost at max capacity.
2) **Chess Blunder-Risk Analyzer:** There is an unhandled scenario when the user may be importing their game data. If the software crashes mid-request, their upload will permanently be stuck in a “running” state. The main issue is that they only allow one request to be in the “running” state, so a timeout mechanism is necessary to circumvent this.